Database architecture overview

1. Purpose

This document defines the logical database architecture for ZAR HORIZON AI @WORK. It translates the Day 8 architecture into a data model that supports the platform, AI Business Partner, and AI Hacker while remaining consistent with the documentation-only scope.

This design is not an implementation and not a founder-approved infrastructure decision. It is a recommended data architecture for future implementation.

2. Database technology direction

The preferred database technology remains PostgreSQL.

The recommended initial database shape is:

- one PostgreSQL database instance for the platform,
- multiple logical schemas for product domains, especially `platform`, `business_partner`, and `ai_hacker`,
- shared tenant-aware tables and identity tables,
- product-specific tables scoped by tenant and workspace,
- strict service-layer and database-level access controls.

This is a hybrid of option A and option B. It gives the team a practical operating model without creating unnecessary database sprawl.

3. Evaluation of option A, B, and C

Option A: One shared database with tenant-aware tables and product-specific data domains

Pros:

- simpler operational footprint,
- lower infrastructure cost,
- easier cross-domain reporting and shared queries,
- easier to manage in a small team,
- adequate for the product maturity level.

Cons:

- broader blast radius if isolation is incorrectly implemented,
- harder to reason about product boundary enforcement,
- risk of accidental cross-product queries if permissions are weak.

Best use:

- early platform stage with strong policy and data governance.

Option B: One database with separate schemas for platform, Business Partner, and AI Hacker

Pros:

- strong logical separation within one PostgreSQL instance,
- simpler backups and operational maintenance,
- easier to share identity and platform tables,
- clearer domain ownership than a single mixed schema.

Cons:

- still requires strict tenant scoping and RLS or service-layer enforcement,
- cross-domain queries need discipline,
- schema boundaries are not security boundaries by themselves.

Best use:

- the best fit for the current architecture and team size.

Option C: Separate databases for each product with shared identity and platform services

Pros:

- strongest physical isolation,
- easier product-level operational independence,
- lower accidental cross-product data leakage risk.

Cons:

- higher infrastructure and operational cost,
- more complexity for reporting and shared platform features,
- more maintenance and backup coordination,
- slower to stand up and harder to operate for a small engineering team.

Best use:

- only when product separation is materially stronger than the current requirements or when security risk is much higher.

4. Recommended initial approach

Recommend: a single PostgreSQL instance with clearly separated logical schemas plus tenant-aware access controls.

This is recommended because it balances:

- strong isolation requirements,
- limited engineering capacity,

- practical deployment complexity,
- manageable backup and restore procedures,
- future product evolution without premature database fragmentation.

The key caveat is that separate schemas are not a security boundary by themselves. Database access controls, tenant scoping, and service-layer checks are still mandatory.

5. Proposed logical database layout

Suggested schemas:

- `platform`
    - users
    - organisations
    - workspaces
    - memberships
    - roles
    - permissions
    - subscriptions
    - plans
    - usage events
    - approvals
    - audit events
    - notifications
    - integrations
    - credentials references
- `business_partner`
    - business_profiles
    - financial_accounts
    - financial_transactions
    - budgets
    - tasks
    - leads
    - customers
    - suppliers
    - quotations
    - analyses
    - recommendations
    - execution states
- `ai_hacker`
    - customer_records
    - assets
    - domains

    - applications
    - api_assets
    - authorisations
    - scopes
    - assessments
    - assessment_runs
    - findings
    - evidence
    - retests
    - remediation_records

- `execution`
    - agent_sessions
    - user_requests
    - execution_runs
    - execution_steps
    - tool_definitions
    - tool_invocations
    - tool_results
    - verification_records

This logical split keeps reporting and domain ownership clear while still sharing core tenant and identity concepts.

6. Data ownership model

The design distinguishes:

- platform-owned shared records,
- tenant-owned product records,
- product-owned domain records,
- temporary execution records,
- sensitive credential references,
- evidence and audit records.

This separation is necessary because shared infrastructure must not imply shared data access.

7. Security implications of the recommendation

The recommended architecture is not security-complete by default. It must be paired with:

- tenant ID on all tenant-owned rows,
- access checks at each API layer,
- database roles with least privilege,
- service-layer validation for every request,
- usage of PostgreSQL RLS or equivalent row constraints where beneficial,
- strong separation of product permissions.

8. Operational implications

This design is manageable for a small team because it keeps:

- an operationally simple PostgreSQL deployment,
- clear product boundaries,
- less backup and restore complexity than multiple databases,
- straightforward cross-product reporting if needed by platform operations.

9. Risks and unresolved questions

The following remain open and require review before implementation:

- whether all tenant-row enforcement will be via service-layer checks alone or a combination of service-layer and PostgreSQL RLS,
- whether product schemas become separate deployments later,
- whether evidence retention and audit retention should be layered separately from operational product data,
- how customer-managed credentials are rotated and expired,
- whether a small number of cross-domain reporting queries are ever needed.

10. Summary

The recommended initial architecture is a single PostgreSQL database with logical domain separation and a strict multi-tenant design. It is the most realistic option for the current team size and product maturity while still supporting the required data isolation and operational boundaries.